

Laboratorio 12: Principios SOLID

Nombre: JOHAN DIEGO HUAMANI SABINA

Fecha: 26 de Julio del 2026

Proyecto: MenteEnCasa

Repositorio: https://github.com/Johansabin/IngSoftware_Proyecto

Rama: main, con trabajo del BC3 integrado desde Bounded-3

Bounded Context trabajado: BC3 - Support Chat

1. Objetivo del trabajo

El objetivo fue aplicar y evidenciar principios SOLID en el bounded context BC3 Support Chat, encargado de sesiones de chat anónimas entre estudiantes y psicólogos. Se revisó la separación por capas, las responsabilidades de dominio, el contrato del repositorio y la forma en que la capa de aplicación depende de abstracciones.

2. Alcance del BC3 Support Chat

- Inicio de sesiones anónimas de chat.
- Envío de mensajes por estudiante o psicólogo.
- Cierre de sesiones activas.
- Consulta de sesiones activas por psicólogo.
- Persistencia en memoria mediante una implementación del repositorio.

3. Avances realizados en GitHub

Avance	Commit	Mensaje	Descripción
1	5d4edf0	feat(bc3): apply SOLID principles to	Refuerzo de principios SOLID en BC3: responsabilidad de reglas en dominio, excepción específica de

support chat

aplicacion y dependencia del servicio hacia el contrato del repositorio.

4. Principios SOLID aplicados

Principio	Aplicacion en BC3	Evidencia
SRP - Single Responsibility Principle	La entidad ChatSession conserva las reglas del agregado, como validar si una sesion esta activa o asignada a un psicologo. El servicio de aplicacion coordina casos de uso y delega creacion, persistencia y reglas al componente correspondiente.	ChatSession.ensureActive, ChatSession.isAssignedTo, ChatSessionAppServiceImpl.createChatSession, createMessage y saveChatSession.
OCP - Open/Closed Principle	La capa de aplicacion queda cerrada a cambios cuando se agregan nuevas implementaciones de persistencia, porque trabaja contra ChatSessionRepository. InMemoryChatSessionRepository puede ser reemplazado por MySQL, JPA u otra implementacion sin cambiar el caso de uso.	ChatSessionRepository define el contrato; InMemoryChatSessionRepository es una implementacion concreta.
DIP - Dependency Inversion Principle	ChatSessionAppServiceImpl no crea ni conoce la infraestructura concreta. Recibe un ChatSessionRepository por constructor y depende de la abstraccion del dominio.	Campo chatSessionRepository y constructor de ChatSessionAppServiceImpl.
ISP - Interface Segregation Principle	El contrato del repositorio expone solo operaciones necesarias para el caso de uso de chat: guardar, buscar por id y consultar sesiones activas por psicologo. No fuerza a la implementacion en memoria a cumplir metodos externos al BC3.	ChatSessionRepository contiene save, findById y findActiveByPsychologist.

5. Fragmentos de código

5.1 SRP: reglas dentro del agregado ChatSession

```
public void sendMessage(Message message) {
    ensureActive();
    messages.add(Objects.requireNonNull(message, "El mensaje es obligatorio"));
    auditData.registerUpdate();
}

public boolean isAssignedTo(UUID psychologistId) {
    return this.psychologistId.equals(
        Objects.requireNonNull(psychologistId, "El id del psicologo es obligatorio"));
}

private void ensureActive() {
    if (!isActive()) {
        throw new IllegalStateException("No se pueden enviar mensajes en una sesion cerrada");
    }
}
```

5.2 DIP/OCP: servicio de aplicacion dependiente de una abstraccion

```
private final ChatSessionRepository chatSessionRepository;

public ChatSessionAppServiceImpl(ChatSessionRepository chatSessionRepository) {
    this.chatSessionRepository = Objects.requireNonNull(
        chatSessionRepository, "El repositorio de chat es obligatorio");
}
```

5.3 SRP: flujo de aplicacion dividido en pasos pequenos

```
public void sendMessage(SendMessageRequest request) {
    Objects.requireNonNull(request, "La solicitud de envio de mensaje es obligatoria");

    ChatSession chatSession = getChat(request.getChatId());
    chatSession.sendMessage(createMessage(request));
    saveChatSession(chatSession);
}

private Message createMessage(SendMessageRequest request) {
    return new Message(request.getContent(), request.getSenderRole());
}

private void saveChatSession(ChatSession chatSession) {
    chatSessionRepository.save(chatSession);
}
```

5.4 Excepcion especifica para el caso de aplicacion

```
public class ChatSessionNotFoundException extends RuntimeException {  
  
    public ChatSessionNotFoundException(UUID chatId) {  
        super("No existe una sesion de chat con el id indicado: " + chatId);  
    }  
}
```

5.5 Repositorio usando comportamiento del dominio

```
for (ChatSession session : sessions.values()) {  
    if (session.isActive() && session.isAssignedTo(psychologistId)) {  
        activeSessions.add(session);  
    }  
}
```

6. Validacion de compilacion

El bounded context BC3 fue compilado localmente con Java 21 usando las capas de dominio, aplicacion e infraestructura:

```
javac -d /tmp/ingsoftware_bc3_classes \  
src/domain/BC3_SoporteChat/*.java \  
src/application/BC3_SupportChat/*.java \  
src/infrastructure/BC3_SupportChat/*.java
```

Resultado: compilacion exitosa, sin errores del compilador.

7. Evidencias recomendadas para insertar como imagenes

- Captura del comando de compilacion anterior ejecutado en terminal.
- Captura del commit 5d4edf0 en GitHub mostrando los archivos modificados de BC3.
- Captura de SonarLint en el IDE sobre los archivos modificados, idealmente sin issues Blocker o Critical.
- Captura de la estructura del repositorio en GitHub mostrando que main mantiene docs y src en la raiz.

Nota: en este entorno no se encontro una herramienta CLI de SonarLint o SonarScanner instalada. La evidencia de SonarLint debe tomarse desde el IDE usado para el curso.